

Modeling and verifying a Radio-Beacons-Based UAV Parcel Delivery Network in UPPAAL

SHIMIAO SUN

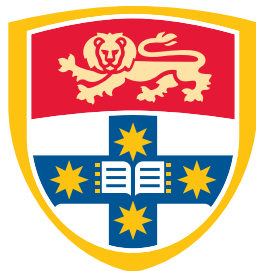
SID: 480142841

Supervisor: Dr. Xi Wu

This thesis is submitted in partial fulfillment of
the requirements for the degree of
Bachelor of Advanced Computing

School of Computer Science
The University of Sydney
Australia

29 May 2025



THE UNIVERSITY OF
SYDNEY

Student Plagiarism: Compliance Statement

I certify that:

I have read and understood the University of Sydney Student Plagiarism: Coursework Policy and Procedure;

I understand that failure to comply with the Student Plagiarism: Coursework Policy and Procedure can lead to the University commencing proceedings against me for potential student misconduct under Chapter 8 of the University of Sydney By-Law 1999 (as amended);

This Work is substantially my own, and to the extent that any part of this Work is not my own I have indicated that it is not my own by Acknowledging the Source of that part or those parts of the Work.

Name: Shimiao Sun

Signature: *Shimiao Sun*

Date: 30th May 2025

Abstract

UAV-based parcel delivery has become a hot topic recently but the application is on a starting stage yet in smart cities. Technically, the exists UAV delivery methods are still on experimental and have limitations, so that the UAV delivery has not been wildly used worldwide. This thesis introduces a new protocol scenario of UAV parcel delivery algorithm without using Global Position System (GPS) navigation, but only use dead-reckoning and signal base station. Therefore, the UAV delivery can be expanded to apply in more different situations in the future. This thesis uses a process calculus for mobile ad hoc network(MANET), which is called ω -calculus, to help build and express the UAV parcel delivery protocol using formal languages and also employ a time-automata model checker UPPAAL, to help model the protocol in time finite state machine and verify correctness and availability of the model. First of all, this thesis designs, analyses and expresses the protocol scenario using network algorithm and ω -calculus. Then, followed by the calculus expression, this thesis uses the modelling tools in UPPAAL to build the time automata of UAV parcel delivery model. After that, this thesis uses Computation Tree Logic (CTL), a temporal logic for specifying system properties, to verify the model includes safety properties, liveness properties and reachability properties. At last, this thesis analyses and describes the limitation of the UAV delivery protocol and how it can be improved based on the further development of algorithm and technology.

Acknowledgements

Special thanks to my thesis project supervisor Dr. Xi Wu for suggesting research direction, guiding me on project scenario, literature references and assistance on problems during our weekly project meetings.

CONTENTS

Student Plagiarism: Compliance Statement	ii
Abstract	iii
Acknowledgements	iv
List of Figures	vii
List of Tables	viii
Chapter 1 Introduction	1
1.1 Motivation of the Project	2
1.2 Contributions	2
1.3 Thesis Structure	3
Chapter 2 Background	4
2.1 Smart city and UAV application	4
2.2 MANET, ω -calculus and Leader-Election Protocol	5
2.2.1 MANET	5
2.2.2 ω -calculus	6
2.2.3 Leader Election Algorithm	7
2.3 Principles of Modeling and Verifying Using UPPAAL	7
2.3.1 Introduction to UPPAAL	7
2.3.2 Modeling	8
2.3.3 Verification	8
Chapter 3 Methodology and Preliminary Result	10
3.1 Introduction of Physical Elements in the Delivery System	10
3.2 Introduction to the delivery process	12
3.3 Expression of the delivery process using ω -calculus	14

Chapter 4	Modeling in UPPAAL	16
4.1	The template of the WareHouse	16
4.2	The Template of the UAV	17
4.3	The Template of the Node	19
Chapter 5	Verification and Formal Analysis	25
5.1	Deadlock-free Property	25
5.2	Reachability Properties	25
5.3	Safety Properties	28
5.4	Liveness Properties	28
Chapter 6	Conclusion	31
	Bibliography	33

List of Figures

2.1	The structure of MANET	6
2.2	Multiple Views of Nodes in MANET Network.	6
2.3	Group-based View of nodes in MANET.	7
3.1	Structure of UAV Delivery System	11
3.2	The drift that will cause in dead reckoning navigation.	12
4.1	The model of WareHouse template in UPPAAL	17
4.2	The model of UAV template in UPPAAL	18
4.3	The model of Node template in UPPAAL	19
4.4	First part of Node template	21
4.5	Second part of Node template	22
4.6	Third part of Node template	23
4.7	Forth part of Node template	24
5.1	The result of executing Deadlock-free Properties	25
5.2	The result of executing Reachability Properties	27
5.3	The result of executing safety properties	29
5.4	The result of executing liveness properties	30

List of Tables

CHAPTER 1

Introduction

As a result of rapid advancement of technology on wireless sensor network and its corresponding wireless smart devices, the development of smart city has enhanced the operational efficiency of cities to provide better services and improved the quality of live for citizens, which includes traffic and transportation services, basic living services, wireless network services and other community services (Okonta and Vukovic, 2024). The explosive growth of population causes rapid urbanization of modern cities, which has promoted pressures on urban infrastructure and other challenges such as traffic congestion, environmental pollution, living stress for citizens and the increasing demand of time for basic living services like parcel delivery. Unmanned Aerial Vehicles are typical efficient all-rounder devices in smart cities. With the support of different types of sensors and wireless sensor network (WSN), UAVs has participated in many services such as traffic monitoring, weather forecast, environmental check and goods delivery. By leveraging the airspace above the infrastructures in smart cities, UAVs do not occupy traffic networks and can have better views for monitoring and data gathering. With the development of UAV movement algorithm, a computer-controlled UAV can have accurate movement on avoiding obstacles and reaching to the specified location with the guide of Global Positioning System (GPS). Mobile Ad Hoc Network (MANET) provides dynamic wireless links communication framework between mobile nodes, which is an ideal network model to support the requirement of UAVs communication in smart cities. MANET allows UAVs to communicate and facilitate efficient data transfer between each other without relying on fixed infrastructure (Romero et al., 2025). However, the application of UAV-based goods delivery in smart cities is still on a starting stage. One of the biggest logistics industry Amazon launched the tests of drone delivery service in 2022, which is called Amazon Prime Air. This service is currently only operating in several small areas with strict restrictions as there are still many technical gaps that need to be solved before fully applying UAV delivery. Here is the summary:

- The cost of UAV delivery is still at a high level. Combined with the payload gap between trucks and cargo UAVs, UAV delivery has very low cost-performance compared with traditional truck delivery.
- The battery life of UAVs is still a problem. Current UAVs have limited flight times due to battery constraints, which means they cannot afford long-distance delivery tasks.
- UAVs may cause serious safety problems. UAVs may easily crash in smart city because of battery failure, collision or adverse weather conditions and will pose risks to public safety and property.
- Legal and security challenges. A legal airspace is necessary for the stable and safety works of UAVs, which needs to be discussed and promulgated by governments. The cyber-attacks and hijacks aimed on delivery UAVs may cause major loss for both companies and customers.
- The development of communication infrastructure such as signal base stations is insufficient, especially in urban areas with signal interference. UAV delivery needs reliable data communication with companies for monitoring and movement fixing to increase delivery location accuracy. Besides, insufficient signal base station may frequently cause communication conflicts because of the network congestion and bandwidth limitations.

1.1 Motivation of the Project

This project is mainly focused on the fifth gap and aims to develop a MANET-based communication framework for UAV goods delivery, present the model of whole delivery process using ω -calculus which is designed expressly to address the MANET modeling problems, and use model checking tools UP-PAAL for modeling and verifying reachability properties, safety properties and liveness properties of this UAV delivery network. The outcomes are expected to provide a solution for MANET-based UAV delivery accurately and efficient communication for data transfer in smart cities.

1.2 Contributions

Here is the summary of the contribution of this work:

- The first to design and implement a UAV delivery protocol in ω -calculus based on the leading of signal base station without using other navigation tools such as GPS.

- Modeling the UAV delivery protocol using UPPAAL. Introduction of the whole delivery process and display how messages transmitting through each node and UAV to accomplish the delivery task.
- Verifying the model of the UAV delivery protocol in UPPAAL to proof the correctness of safety properties, liveness properties and reachability properties to make sure the project is deliverable.

1.3 Thesis Structure

The whole thesis is organized as follows: In chapter 2, we introduce and review the background of the MANET and the application of UAVs in smart cities, a process calculus named ω -calculus for designing and building the UAV delivery protocol, and the model checking tool UPPAAL for modeling and verifying the UAV delivery protocol. In chapter 3, we describe the physical elements and the whole delivery process, then we use ω -calculus to express and design the delivery protocol for further modeling use. In chapter 4, we describe the model of UAV delivery system in UPPAAL and explain the time automata. Chapter 5 presents the verification of the system in UPPAAL using formal language Computation Tree Logic (CTL). Chapter 6 discusses the conclusion of the project and the directions of future work to make the scenario complete for real world application.

Background

2.1 Smart city and UAV application

Smart city is an urban area which uses advanced technology such as sensors, internet and devices to improve the quality of live experience for citizens. In a survey of UAV applications in future smart cities, it mentions the UAV-based service of merchandise order delivery (Mohamed et al., 2020). UAVs can handle the missions of delivering customer orders within a short time because of the flexibility of movement in the sky. Compared with traditional delivery method using trucks, UAVs are flexible to move fast and can reach almost every corner of cities for rapid delivery even in crowded cities. Many delivery companies and online shopping companies, included Amazon and Google, have started the experiment of UAV delivery in a small range of area to find out the feasibility and sustainability of UAV delivery. In addition, other applications based on UAV in smart cities such as UAV health emergency services and infrastructure inspections, are also mentioned in this survey, to provide interesting research fields to help come up with project scenarios. However, since the application of UAVs in smart cities is still in the beginning stage and the technology is being explored, there are many limitations that need to be pointed out. First of all, the hardware technology of civilian UAV is not enough to handle complicate tasks such as parcel delivery. Restricted by the body size of the UAV, the battery capacity is one of the most important issues that needs to be solved. Maintaining long distance flying always consumes a large amount of electricity. Moreover, UAV also contains many sensors with different functionalities such as gyroscope, signal receiver and even navigator to support complex tasks. Thus, electric blackout may happen frequently in one long distance flying without supply. Secondly, employing UAVs in smart city can also occur safety problems. As mentioned in hardware issues, the civilian UAVs may cause blackout during the delivery mission. An uncontrolled crash in smart city can cause significant casualties and property damage. In addition to hardware issues, there is still no mature legal provisions to focus on the flying of UAVs in smart city at the present stage. Traffic accidents can also occur between UAVs

and the airline of the UAVs should also been set to avoid collision with architectures, citizens and other UAVs. Irregular departure and landing of UAVs can also cause damage to city residents. Thirdly, the cost of using high-tech UAVs for parcel delivery is much higher than traditional parcel delivery using mainstream transportations.

2.2 MANET, ω -calculus and Leader-Election Protocol

2.2.1 MANET

A Mobile Ad Hoc Network (MANET) is a network of autonomous mobile nodes connected by wireless links (Figure 2.1). Most of the mobility devices with the functionality of signal broadcast and receiving that can link and join the network can become an autonomous mobile node such as mobile phones, computers (laptops), online payment pos machine and UAVs in smart city (Singh et al., 2010). Each node in the network has a physical transmission range so that a node can transmit messages to another node is and only if their locations are in each other's physical transmission range (Figure 2.2). These two nodes can be called neighbouring nodes if they can send communication message to each other. However, autonomous mobile nodes can move flexibly in MANET, which can cause a rapid change of neighbourhood relationship between nodes and also lead to the change of topology structure of the whole network. Here is the conclusion of features in MANET:

- Autonomous Mobile Nodes: it refers to nodes that are not fixed and can move flexibly in network.
- Wireless Links: nodes in network communicate with each other over wireless links, typically have broadcast, multicast and point-to-point communication.
- Local Broadcast Communication: a message is sent from one node to all nodes within physical communication range (every neighbouring node).
- Multicast Communication: a message is sent from one node to a selected group of nodes which belongs to its neighbouring node list.
- Point-to-point Communication: a message is sent from one node directly to another specified node which belongs to its neighbouring node list.
- Dynamic Topology: the topology structure of the network can change frequently due to node mobility (Hassija et al., 2021).

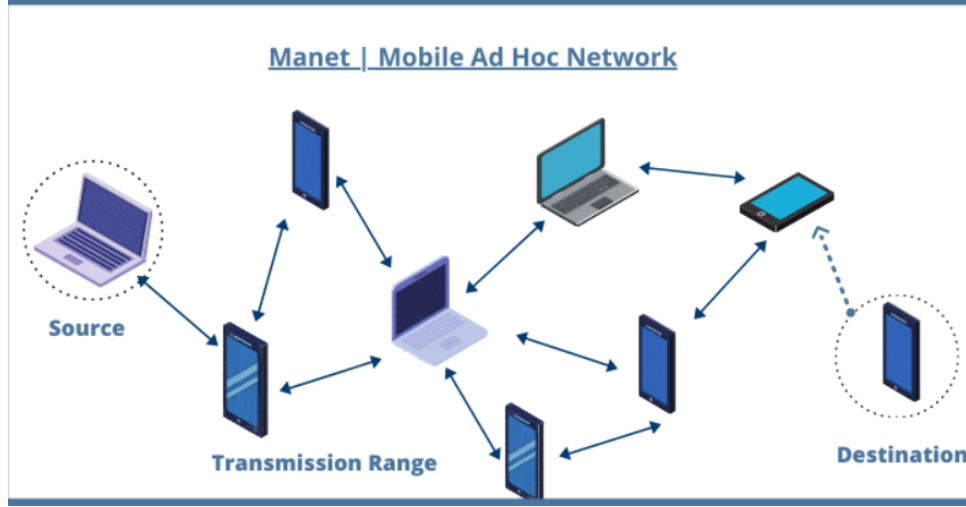


FIGURE 2.1: The structure of MANET



FIGURE 2.2: Multiple Views of Nodes in MANET Network.

2.2.2 ω -calculus

ω -calculus is a process calculus for formally modeling and reasoning about MANET and their protocols provided by Anu Singh, C. R. Ramakrishnan, Scott A. Smolka in 2009. This process calculus can be used to represent project scenarios described this in formal language. The key feature of ω -calculus is the separation of nodes communication, which is suitable to represent the model of communication between UAVs and signal base stations. The ω -calculus is introduced to describe and cover main characteristics and operations of MANET-based network model, which includes local broadcast communication, node mobility and node communication behaviour in network topology. In ω -calculus specification, nodes communicate via groups (Figure 2.3) (Singh et al., 2010). A Group means that the communication

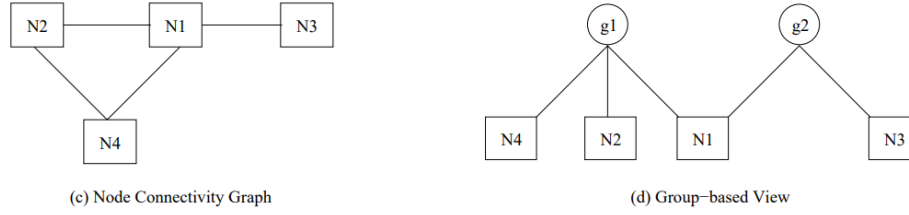


FIGURE 2.3: Group-based View of nodes in MANET.

range of Nodes in the same group are overlapping and they can communicate with each other. Hence, local broadcast can also means a process sends a message to all processes within the same group.

2.2.3 Leader Election Algorithm

Leader-election protocol is the algorithm of elect the node with the maximum id in a group of nodes. Those nodes are distributed in Euclidean planar settings and only nodes in each other's connection range can communicate with each other. This model described how the communication data passes between nodes and how can a node communicate with another nodes which is not in its neighbors (Singh et al., 2010). Applied in UAV delivery system, it can be used to show how UAV can get communicate with the target signal base station and how the signal base station sent the data message back to the warehouse.

2.3 Principles of Modeling and Verifying Using UPPAAL

2.3.1 Introduction to UPPAAL

UPPAAL is a toolbox application for modeling, simulation and verification of real-time systems, which is developed by Uppsala University and Aalborg University. Technically, it is mainly used to analyse real-time systems where timing constraints are crucial (Lv et al., 2021). Basically, modeling and verifying using UPPAAL can be divided into three main steps. First of all, designing and modeling a real-time system using timed automata with integer variables, structured data types and channel-based communication. The channel-based communication here can be both local broadcast communication and unicast communication. Then, the UPPAAL toolkit has the simulation to explore the behaviours of the model based on the time line. Thirdly, verifying the model system properties using query language such as Computation Tree Logic (CTL) (Behrmann et al., 2004).

2.3.2 Modeling

In UPPAAL, each process can be modeled as a timed automaton (TA). Formally, A timed automaton is described as a tuple: $TA = (L, l_0, C, A, E, I)$, where:

- L : a set of locations (states).
- l_0 : the initial location which is belong to L .
- C : a set of clocks.
- A : A set of actions (synchronization labels)
- $E \subseteq L \times A \times B(C) \times 2^C \times L$: a set of edges (transitions), each with guard, reset and target.
- $I: L \rightarrow B(C)$: assigns invariants to locations (clock conditions that must hold while in a location)

Here, clocks simulates the real time pass and all clocks set in the same time model increase at the same time, which means the clock variable cannot be modified respectively and can only be reset to zero during the transitions between two locations (states) (Behrmann et al., 2004). Each location has a property named Invariants which limit how long the process can stay in a location. For example, the invariant clock $x \leq 10$ means the system will jump out this location and move on to the next location when the time x is greater than 10 time units.

2.3.3 Verification

UPPAAL also provide tools for verification via model checking of real-time system. The main purpose of a model checker is to check whether this model can meet the requirement specification (Behrmann et al., 2004). UPPAAL allows users to use query languages such as CTL to check their model. The query language contains both path formulae and state formula, where:

- **State Formulae:** A state formulae is used to evaluate the property of a state without looking at the behaviour of the whole model.
- **Path Formulae:** A path formulae is used to describe how state formulae evolve over time guard.

In UPPAAL, deadlock represents that there is no further progress possible for the system, that is, the system will finally get stuck and terminate at one state. The formula has the key word deadlock and is satisfied for all deadlock states. Three kinds of properties are applied in UPPAAL for model checking, they are:

- **Reachability Properties:** Reachability Properties asks “in this system, is it possible to reach a certain state”, which can be expressed using a simple path formulae: $E\langle\rangle\phi$. Here, E means “there exists a path”, $\langle\rangle$ means “eventually”, and ϕ is a state formulae. Thus, this path formulae means that “There exists at least one execution path where the state formulae ϕ eventually holds (become true)”. Reachability Properties are mainly used for sanity check while designing a model (Behrmann et al., 2004).
- **Safety Properties:** Safety Properties asserts that “something bad will never happen” during system execution, which can be expressed using a simple path formulae: $A[]\phi$. Here, A means “for all paths”, $[]$ means “in all states along the path”, and ϕ also means a state formulae. Thus, this path formulae means that “In every possible execution path, at every point in time, ϕ is always true”. In UPPAAL, these properties are formulated positively (Behrmann et al., 2004).
- **Liveness Properties:** Liveness Properties asserts that “something good will eventually happen” in the system’s future, which can be expressed using a simple path formulae: $A\langle\rangle\phi$. Here, A means “for all paths”, $\langle\rangle$ means “eventually”, and ϕ also means a state formulae. Thus, this path formulae means that “On every execution path, ϕ will eventually holds (become true)”. Response Properties are a form of Liveness Properties, which can be expressed as “ $\phi \rightarrow \psi$ ”. Here, both ϕ and ψ are state formulae. Thus, this path formulae means that “every time when ϕ happens (become true), ψ will eventually follow (become true)” (Behrmann et al., 2004).

Mathodology and Preliminary Result

3.1 Introduction of Physical Elements in the Delivery System

This figure shows the structure of UAV delivery system (Figure 3.1). The components of the UAV delivery system can be described as follows:

- Warehouse, which can be denoted as WH , is the place with lots of parcels that are waited to be delivered. The delivery company will have a warehouse in each suburb and the parcels concentrated in this warehouse will be delivered to the customers whose delivery address is in this suburb.
- Delivery UAV, which can be denoted as U_x (UAV-X), will carry the package from the warehouse and deliver the package to the customer's address.
- Ground signal base station, which can be denoted as N_x (Node x), are a part of ground-based infrastructure and are mainly used to receive and transfer data and message between delivery UAVs and warehouse.
- Aerial signal base station, which can be denoted as N_x (Node x), are Helium balloon airship characteristics with ropes traced and fixed on the ground. The aerial signal base stations have the same functionalities as ground signal base stations.
- Customer's home, which can be denoted as H_x (Home X), can also represent the final destination of the delivery UAV.

Both aerial base stations and ground base stations will use Wi-Fi to communicate with each other, and delivery UAVs will apply Bluetooth as the communication tool with both aerial base stations and ground base stations. Technically, the aerial base stations are much more flexible than ground base stations. Therefore, they are mainly used to supplement the empty space of the signal base station matrix (Chandrasekharan et al., 2016). The effective connection distance for Wi-Fi will be around 100 meters.

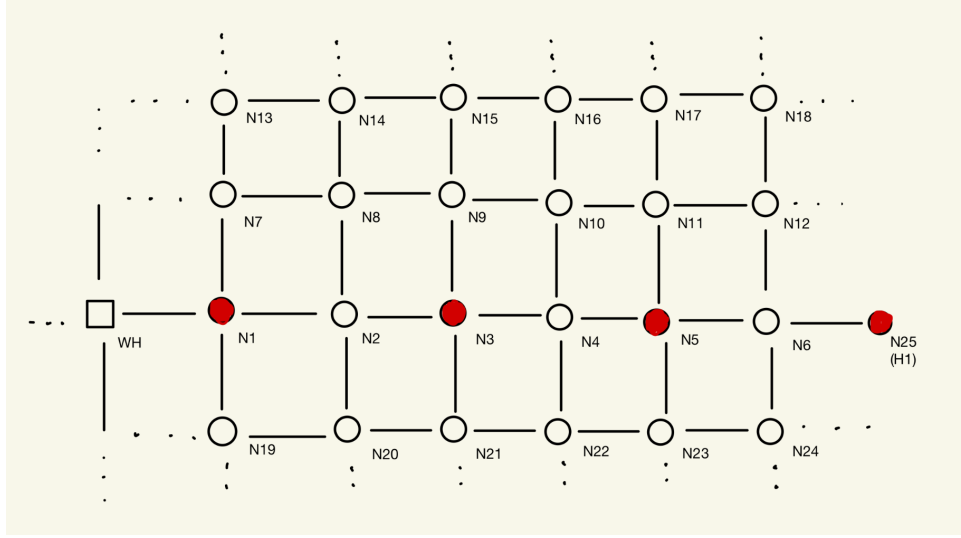


FIGURE 3.1: Structure of UAV Delivery System

Hence, the distance between two neighbouring nodes in this signal base station matrix is 100 meters. However, Wi-Fi is not power-friendly and can also cause battery failure for UAV since UAV needs power for long-distance flying. Compared with Wi-Fi, Bluetooth is much more power-friendly for UAV but the effective connection range is only about 10 meters. Thus, the Bluetooth receivers are also applied on both aerial base stations and ground base stations to make sure the deliver UAVs can communicate with the signal base station system. Customers can use home Wi-Fi with Bluetooth receiver, to make the delivery UAV get connected with the MANET to receive parcels and for further operations. We can assume that the deployment of signal base stations and delivery UAVs are much higher than architectures in smart city so that collision problems and avoiding obstacles can be ignored for accurate movement.

In order to save power for UAV and gain more airborne time, the UAV will apply dead reckoning navigation rather than using GPS navigation. Dead reckoning is a navigation technique used to plan route based on the beginning location and the destination location. In this system, each signal base station will have GPS sensor with their location recorded using coordinate system (x, y) . Thus, the heading and distance of UAV movement can be calculated. Then, we can consider the movement speed of UAV is uniform motion and the value of UAV speed can be measured using speedometer (Rashid and Turuk, 2015). Hence, the moving time from a node to another node can be predicted. However, the limitation of dead reckoning cannot be ignored. During the moving period, the UAV can be effected by environment elements especially wind. Without the help of GPS, the UAV cannot correct the airline and may cause significant drift (Figure 3.2). Based on this research, the deviation of small and medium

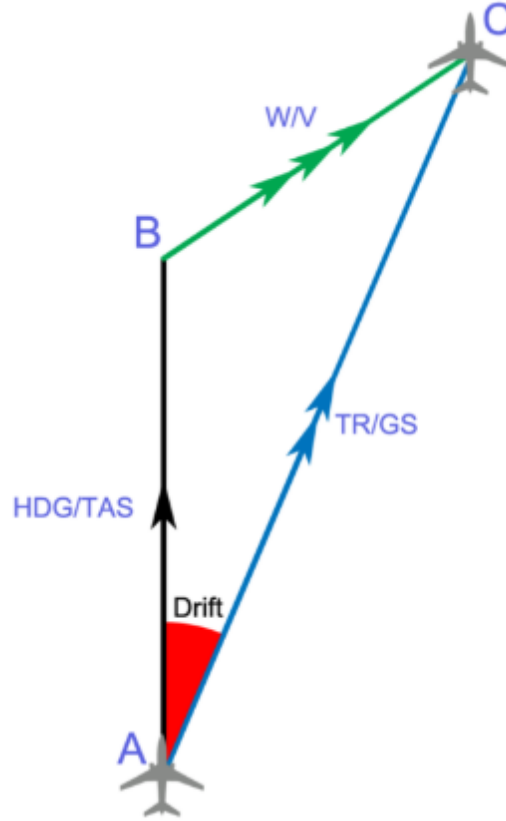


FIGURE 3.2: The drift that will cause in dead reckoning navigation.

UAV using dead reckoning can become 2% to 5%. Hence, in order to make sure that the UAV can still get connection with the next node, we need to control the drift and make it become less than 10 meters. Thus, because the largest connection range of Bluetooth is 10 meters, the UAV can still get connecting with the next node even the deviation is at 5%. Thus, we can calculate that the distance between two initial nodes will be good to set as 200 meters. Since the distance between two nodes in this system is 100 meters, the Manhattan Distance of the initial nodes for UAV's destination cannot be over 200 meters.

3.2 Introduction to the delivery process

A node may receive five kinds of message during the whole delivery process:

- Finding Message: Finding Message will be denoted as *find_msg* in UPPAAL and this message will be broadcast from the initial node to all its neighbouring nodes.

- Error Message: Error Message will be denoted as *error_msg* in UPPAAL and this message will be used to report *NodeNotFoundError* if the target node cannot be found.
- Acknowledge Message: Acknowledge Message will be denoted as *ack_msg* in UPPAAL and this message will be used to report that the target node has been found.
- Location Message: Location Message will be denoted as *loc_msg* in UPPAAL and this message will be used to pass location information of target node.
- UAVState message: after the UAV received the direction and the destination and start the movement to the next signal base station, the initial node will send this message with UAV's id (uid) back to *WH* so that *WH* can keep track on the movement and state of the delivery UAV.

Based on the Figure 3.1, the delivery UAV will start from the warehouse carried with the parcel and a list of signal base station id l_1 . The list of target node's id, which can be denoted as *tid*, is provided by warehouse and shows the signal base stations that the delivery UAV will pass to reach the destination of customer's home. As in this example, the UAV U_1 need to deliver the parcel form *WH* to the home H_1 , which is denoted as H_1 . Thus, it needs to reach and pass N_1 , N_3 , then N_5 until it successfully reaches N_{25} . The computer in warehouse will read l_1 and lead the UAV U_1 to the first location N_1 . Basically, since every signal base station has a coordinate location (x, y), *WH* can calculate the direction and distance of N_1 and send them to U_1 , so U_1 can follow the guide and reach N_1 . Once U_1 reaches and communicates with N_1 , U_1 will send the *tid* (N_3 's id) from l_1 to N_1 and N_1 will become the initial node and start the process of *ProcessFindNode* to communicate and find the location of N_3 . This process will be introduced in ω -calculus in the next subsection. Here in this example, all of N_1 's neighbouring nodes N_2 , N_7 and N_{19} will receive the broadcast message *find_msg* and build a private connection channel using N_1 's id. This channel will be used for passing *ack_msg* and *error_msg*. Every node (N_2 , N_7 and N_{19}) that receives the broadcast *find_msg* will compare with the *tid*. If their own id matches the *tid*, the node that matches the *tid* needs to send the *ack_msg* and the *loc_msg* carrying the information of target node's location back to N_1 using the private unicast channel. After N_1 receives the *ack_msg* and the location, N_1 can calculate the heading and distance using *loc_msg* and send the information back to the UAV U_1 . U_1 can follow the heading and the distance and move to the next target node. If all of them (N_2 , N_7 and N_{19}) are not target node, they will broadcast the *find_msg* to their neighbouring nodes and start the finding process again. After two iterations of finding nodes, if the target node has been found, the UAV U_1 will reach to the target node. If the target node cannot be found, the node will report *error_msg* back to its parent node (the node which broadcast *find_msg* to it) using the private unicast channel. Once the initial node (the node which get connected with the UAV) receives the

error_msg, it will directly report the error message to *WH* and terminate the whole process. In this example, N_3 is the target node and the UAV U_1 will move to N_3 and start a new finding node process. Here, N_3 will become the new initial node and broadcast the *find_msg* to all its neighbouring nodes, and the *find_msg* here contains N_3 's id and the new *tid* from the node list l_1 (which is N_5 in this example). This process will be circulated until it reaches N_{25} (H_1). The customer will receive the parcel with a series of receiving process and after the delivery process is finished, the UAV will reverse the signal base station node in l_1 and follow by the process again until it returns to the warehouse *WH*.

3.3 Expression of the delivery process using ω -calculus

Thus, we can determine the instance of node process using ω -calculus:

$$\begin{aligned} & node(id, isFound, init, find, tid, pChan, WHChan) \stackrel{\text{def}}{=} \\ & r(finding(tid)).processFindNode(id, isFound, init, find, tid, pChan, WHChan) \\ & + r(ack(tid), location).processAck(id, isFound, init, find, tid, pChan, WHChan) \end{aligned}$$

In this calculus, $r(finding(tid))$ means the node receives the finding message with the input *tid*. Then, it reaches the *processFindNode* state where it broadcast the finding message and goes to state *awaitAck*. Here is the summary of parameters in this calculus:

- *id*: the id of the node.
- *isFound*: the boolean represents whether the target node has been found.
- *init*: the boolean represents whether the node is the initial node.
- *find*: the boolean represents whether this node has participated in the finding process.
- *tid*: the id of target node.
- *pChan*: the parent channel of the node. the node which broadcasts the find message is the parent node and nodes which receive the finding message is the child node.
- *WHChan*: the channel that is used to unicast the warehouse *WH*.
- *location*: the location information of the target node.

$$\begin{aligned}
& processFindNode(id, isFound, init, find, tid, pChan, WHChan) \stackrel{\text{def}}{=} \\
& [id! = tid]b(finding(tid)).awaitack(id, isFound, init, find, tid, pChan, WHChan) \\
& + [id == tid]pChan(ack(tid), location).processAck(id, isFound, init, find, tid, pChan, WHChan)
\end{aligned}$$

In the *processFindNode*, the initial node will broadcast the finding message with *tid* to the neighbouring nodes. Each node will receive that finding message, mark the initial node as their parent node and keep the channel of the initial node as *pChan*. They will compare their own id with the *tid* in the finding message. If it matches, it will unicast the acknowledge message (*ack_msg*) and location message (*loc_msg*) to its parent node stored in *pChan* and then reach to *processAck* with the Boolean *isFound* is true. If it does not match, it will broadcast the finding message with the *tid* and then reach to the state *awaitAck*. A node in *awaitAck* state may receive an *ack_msg* and reach *processAck* state or cannot reach *processAck* state if every node is traversed but no one matches. In the previous case, if the *init* Boolean is false, that is, the node is not the initial node, the node will unicast *ack_msg* with *tid* and location message to its parent node via *pChan* until the messages reaches the initial node. If *init* is true, that is, the node is the initial node, the initial node will calculate the distance and direction between start node and target node using GPS coordinate point and unicast the calculated location message to the delivery UAV. Then it will reach the *processReportState* to report the state of delivery UAV to *WH*. In the latter case, the initial node will unicast the error message back to *WH* using *WHChan*, which will be discussed later in this section.

$$\begin{aligned}
& awaitAck(id, isFound, init, find, tid, pChan, WHChan) \stackrel{\text{def}}{=} \\
& [init == false]pChan(ack(tid), location).processAck(id, isFound, init, find, tid, pChan, WHChan) \\
& + [init == true]pChan(calculatedlocationmessage).processReportState(id, uid, WHChan) \\
& processReportState(id, uid, WHchan) \stackrel{\text{def}}{=} WHChan(UAVState(uid)).endProcessFindNode()
\end{aligned}$$

A node can broadcast the finding message with *tid* to all nodes connect with it with the boolean *find == false*. If no node can receive the finding message, it is clear that this node is at the end of this spanning tree and will report a notification *NodeNotFound* back to the initial node. If the initial node received every *NodeNotFound* message from the leaf of the spanning tree but still no location message received, it will reach the *processReportErrors* and report the error message *NodeNotFound* to *WH*.

Modeling in UPPAAL

In this chapter, I formally model the UAV Delivery System in UPPAAL. The model of the system has three templates:

- WareHouse is the model of warehouse (WH), shown in Figure X.
- UAV is the model of the delivery UAV (U_1), shown in Figure X.
- Node is the model of the signal base stations, shown in Fig X. since the template Node is complicate contains with multiple situations and functionalities, we will separate the model and discuss them in different situations, which can make it easier to get the model clear and correct.

The project itself also contains global declaration and template declaration contains with parameters, variables and assist functions for perfect functionalities simulation. Thus, we can discuss them separately in each template.

4.1 The template of the WareHouse

This template shown in Figure 4.1 has three locations: *Idle*, *ProcessReport* and *ProcessError* and *Idle* is the initial location. The warehouse WH may stay in this location after the delivery UAV U_1 leaving WH for the parcel delivery task and wait for further information of delivery process from signal base stations. The *Idle* location does not have invariant, which means that WH can stay in this location for unlimited time until it receives further information from corresponding channels. If WH receives *rep_msg* from a signal base station that to report the delivery UAV has found the next node and has left the location of current signal base station, it will transit from location *Idle* to location *ProcessReport* to report this information to the delivery company, so that the company can track the location of U_1 immediately. And then going back to *Idle* again to wait for the next UAV state information. If WH

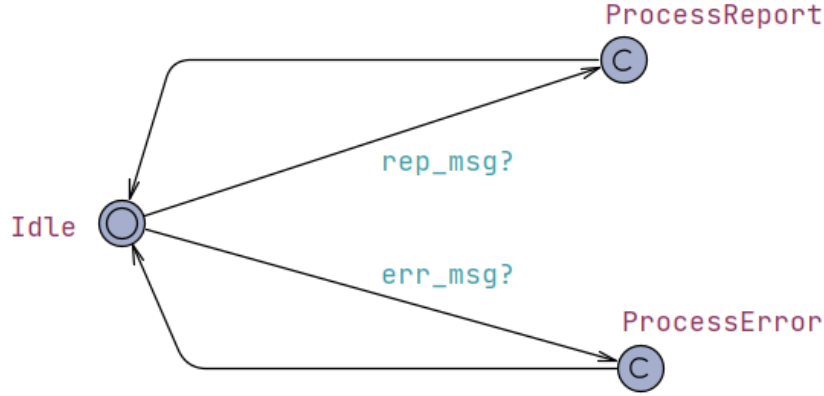


FIGURE 4.1: The model of Warehouse template in UPPAAL

receives *err_msg* from a signal base station that to report U_1 does not found its next location node, it will transit from *Idle* to the location *ProcessError* and report the error message to the delivery company, so that the company can know the node list l_1 is wrong. Then, WH will go back to *Idle* again to standby.

4.2 The Template of the UAV

This template shown in figure 4.2 has five locations: *Idle*, *awaitAck*, *ProcessAck*, *Moving* and *Terminate*. *Idle* is the initial location. Here both *processAck* and *Moving* location are committed locations, which means that the system must leave this location immediately and no time delay is allowed in this location. That is, in the system with multiple automaton entities, if the location of one automaton entity is in a committed location, the next transition must be the transition that leave this committed location and transit to the next location. Here, both *processAck* and *Moving* do not allow time delay and U_1 should leave the location *Moving* and transit to *Idle* immediately. However, the moving of U_1 from the initial node to the next node takes time based on the distance of two nodes. I still set the *Moving* as committed location because the details of how U_1 moves between nodes is not important to

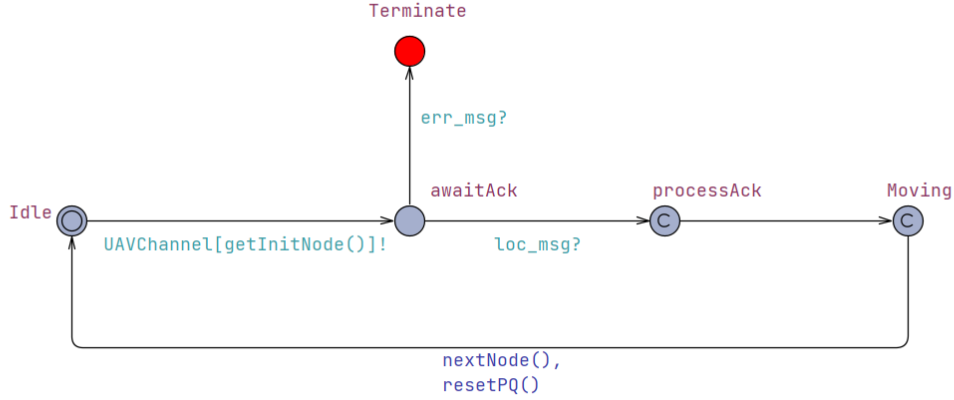


FIGURE 4.2: The model of UAV template in UPPAAL

be considered for delivery process modeling and simulation. It is easier to think and model if U_1 leave the initial node and reach the next node immediately. Besides, it can also make the simulator execute quicker and more flexible. Back to the main process, U_1 reaches the initial node (signal base station) and get connected with the initial node using the channel $UAVChannel[id]$. The id here is the initial node id, which means that U_1 has a list of channels that is used to unicast with every initial node. When U_1 communicates with initial node and initial node starts a finding process, U_1 will stay in the *awaitAck* location for further messages communication. If the initial node sends back the location message of the next node which contains heading and distance, U_1 will transit to *processAck* and then to location *Moving*. Then it will reset the variable and change the target node id to the next node followed by the list l_1 and transit to *Idle* again to communicate to the next initial node. However, if the target node cannot be found and an *err_msg* passes to U_1 by initial node, U_1 will transit to terminate location and land to the platform of the signal base station to wait for further command from the delivery company. In that case, since the company can receive the id of the initial node, they can send people to the location of U_1 and recycle it.

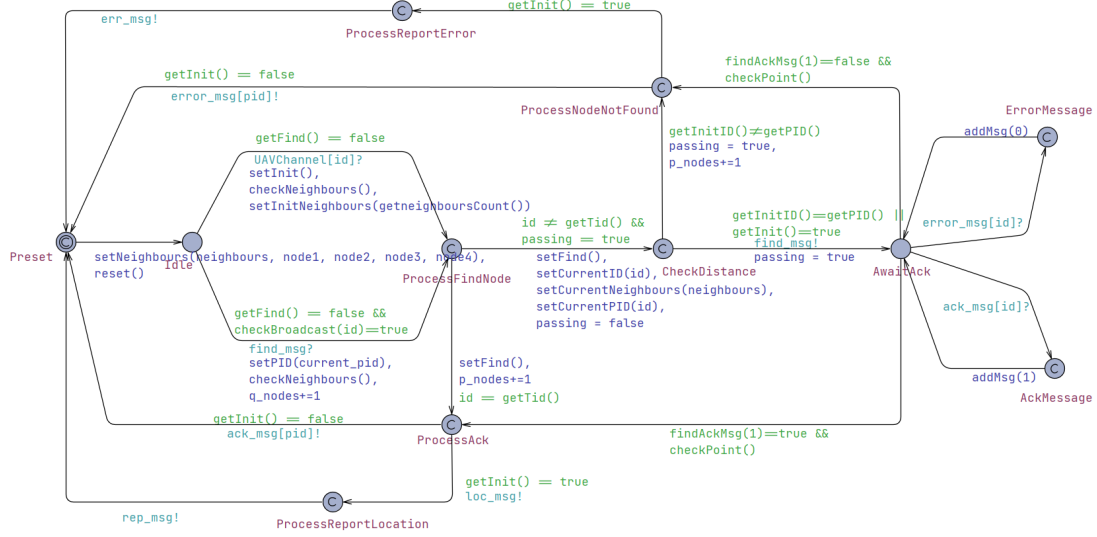


FIGURE 4.3: The model of Node template in UPPAAL

4.3 The Template of the Node

The Node template shown in Figure 4.3 is very complex and I am going to divide it into several small parts based on different situations to make it clear. Before discussing the model, it is necessary to discuss about the parameters and variables in Node template. Here is the summary:

- *initial*: the boolean that represents whether this node is the initial node.
- *find*: the boolean that represents whether this node has participated in the finding process.
- *pid*: the id of the node's parent node, that is, the node which broadcast the *find_msg* will become the parent node, and nodes which receives the *find_msg* will become the child nodes.
- *int msglist[4]*: a 4 sizes integer array which stores the type of message receives in location *AwaitAck*. If the node receives the *ack_msg* from another node, it will store 1 in the *msglist*. If the node receives *err_msg* from another node, it will store 0 in the *msglist*.
- *int neighboursCount*: represent the number of neighbouring nodes that this node has.
- *neighbour_t neighbours*: *neighbour_t* is a type definition structure and can be used as an array to store neighbouring nodes id. *Neighbours* is the name of the array.
- *const int[1, N] id*: represents the id of the node.
- *int node1, int node2, int node3, int node4*: represents the id of neighbouring nodes.

This figure (Figure 4.4) shows the first part of the Node template. Here, we are going to mainly discuss three locations and their transitions: *Preset*, *Idle* and *ProcessFindNode*. *Preset* is the initial location, which is also committed. Before U_1 starts the delivery process from WH , each node needs to be prepared and reset every variable to default, which contains their neighbouring nodes and that is what the function *reset()* does. Since the signal base stations have fixed location, their neighbouring nodes will not change once it has been set. Thus, the function *setNeighbours(neighbours, node1, node2, node3, node4)* is to set neighbouring nodes for every node in the system. The purpose to store the neighbouring nodes is to simulate the distance in UPPAAL model. Based on the definition of broadcast, every node in the range can receive the broadcast message. However, UPPAAL does not have the distance restriction, so that I apply a list of neighbouring nodes for simulation that only nodes in the neighbouring list can receive the broadcast message. Then, nodes will transit from location *Preset* to location *Idle* and wait for communication from U_1 or *find_msg* from another node. On the one hand, if a node receives the communication message from U_1 and this node has not participated in the finding process (which can be checked using the guard *getFind() == False*), this node will become the initial node and the function *setInit()* is used to set this node as the initial node. Meanwhile, the function *checkNeighbours()* will check how many neighbouring nodes does this node have, so that we can use the function *setInitNeighbours()* to store in the system. After that, the initial node will transit from *Idle* to *ProcessFindNode*. On the other hand, if a node has not participated in the finding process and receives *find_msg* from another node, this node will become a child node and we use the function *setPID()* to store the parent node's id, which will be used in unicast to communicate with its parent node. after that , the location of the node will be transit from *Idle* to *ProcessFindNode*.

In the second part of the figure (Figure 4.5), when the node is at the location *ProcessFindNode*, it means the node has participated in the finding process. Hence, the boolean variable *find* will be set as *true* using function *setFind()* so that the node does not need to reply the broadcast message *find_msg*. Here, the node will check whether its id is equal to the *tid*. If the *tid* matches its own id, the node will transit from location *ProcessFindNode* to *ProcessAck*. Then, the node in *ProcessAck* will check whether this node is the initial node. Here, it has two different transition ways. If the node is the initial node, the system will transit from *ProcessAck* to *ProcessReportLocation* and unicast *loc_msg* to U_1 . Then, the initial node will unicast *rep_msg* to WH and transit back to the initial location *Preset*. If the node is not the initial node, it will unicast *ack_msg* to its parent node and then transit from *ProcessAck* to *Preset*.

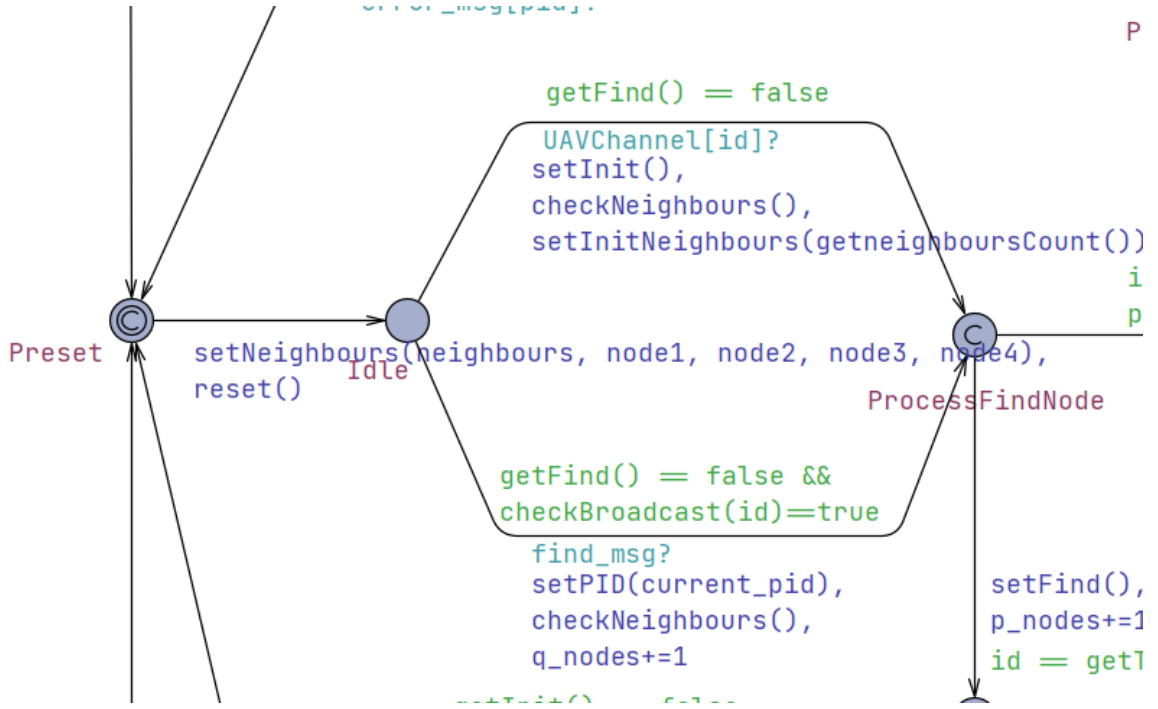


FIGURE 4.4: First part of Node template

In the third part of the node template (Figure 4.6), we go back and start from the location *ProcessFindNode*. If the node's id does not match *tid*, it will transit from location *ProcessFindNode* to location *CheckDistance*. During the transition, there are three variables that need to be updated.

- The function *setCurrentID(id)* is used to store current node's id to the global variable, and this id may become a parent node id for its neighbouring nodes.
- The function *setCurrentNeighbours(neighbours)* is used to store current node's neighbouring nodes in global variable.
- The function *setCurrentPID(id)* is used to set the id which stores in global variable as the current node's *pid*.

The purpose of location *CheckDistance* is to check the distance between current node and initial node. Basically, the Manhattan Distance between target node and initial node cannot be greater than 200 meters because of the dead reckoning navigation. Thus, the target node cannot exist after two times *find_msg* broadcast. Hence, if current node's *pid* is not equal to the initial node's id and the target node is still not found, we can say that the target node cannot be found in this branch, and the current node can transit from location *CheckDistance* to location *ProcessNodeNotFound*. Then, if the current node is not the initial node, it will unicast the *error_msg* back to its parent node using channel

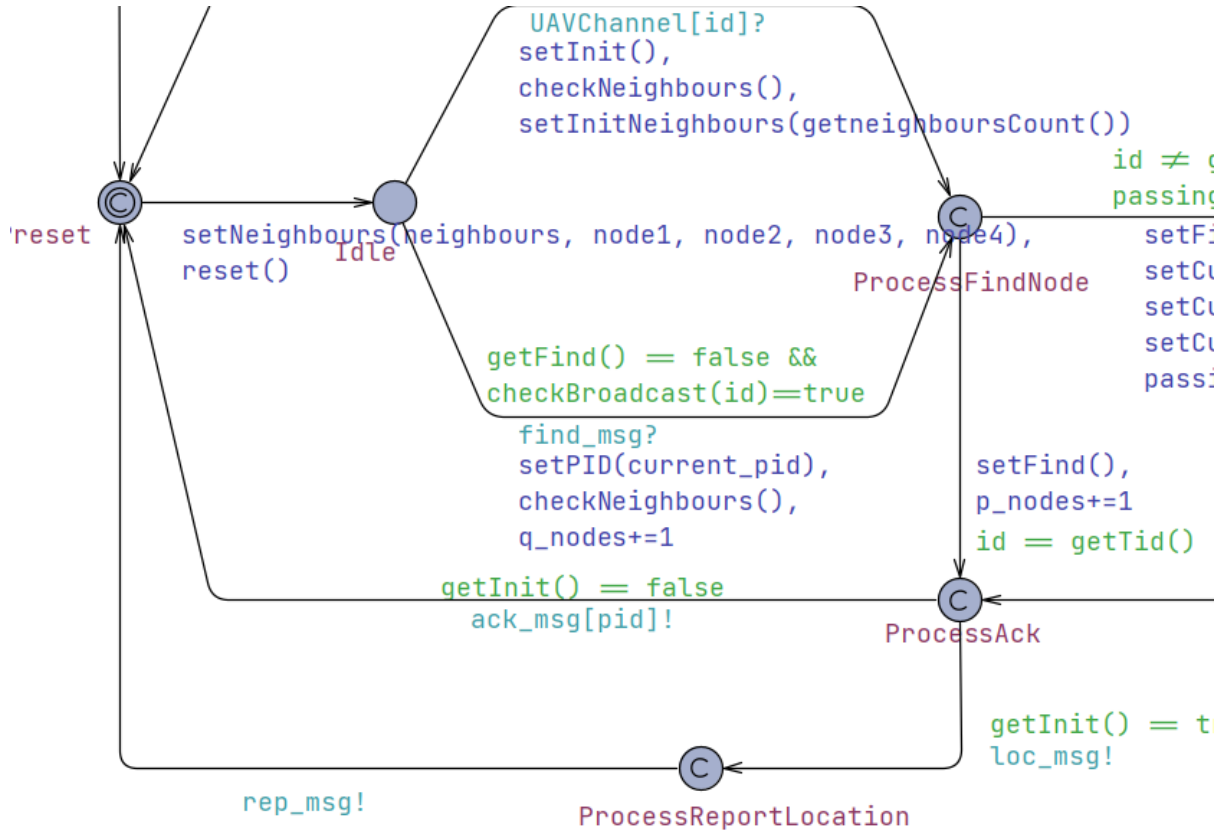


FIGURE 4.5: Second part of Node template

synchronization *error_msg[pid]!* and then going back to location *Preset*. If the current node is the initial node, it will transit to location *ProcessReportError* and unicast the *err_msg* back to *WH* using channel synchronization *err_msg!*.

In the forth part of the node template (Figure 4.7), we go back and start from the location *CheckDistance*. If the check distance passed, that is, the current node is the initial node or the current node's parent node is the initial node, it will broadcast the message *find_msg* using channel synchronization *find_msg!* and transit to location *awaitAck*. Here, the current node will wait for both *error_msg* and *ack_msg* from child nodes. After all child nodes reply to it, the current node will check the message list. In location *awaitAck*, if a child node sends back *error_msg* (target node not found in its branch), it will add 0 to *msg_list* using function *addMsg(0)*. If a child node sends back *ack_msg* (target node has been found in its branch), it will add 1 to *msg_list* using function *addMsg(1)*. After every child node

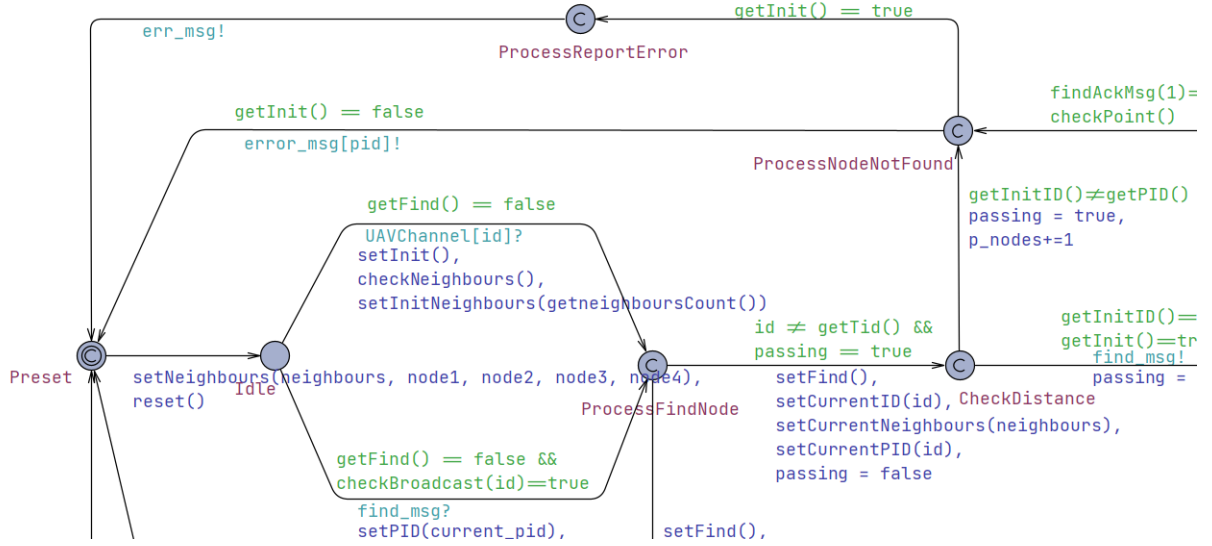


FIGURE 4.6: Third part of Node template

replies, we can check the numbers in `msg_list`. If there exists 1 in `msg_list`, we can say that the target node is found and the current node can transit to location *ProcessAck*. If there does not exist 1 in `msg_list`, we can say that the target node is not found and the current node can transit to location *ProcessNodeNotFound*. The rest of transitions to location *Preset* are the same as we discussed above. After the UAV U_1 receives the location message and moves from initial node to target node, all nodes will transit from location *Preset* to location *Idle* and wait for the next iteration of finding node process.

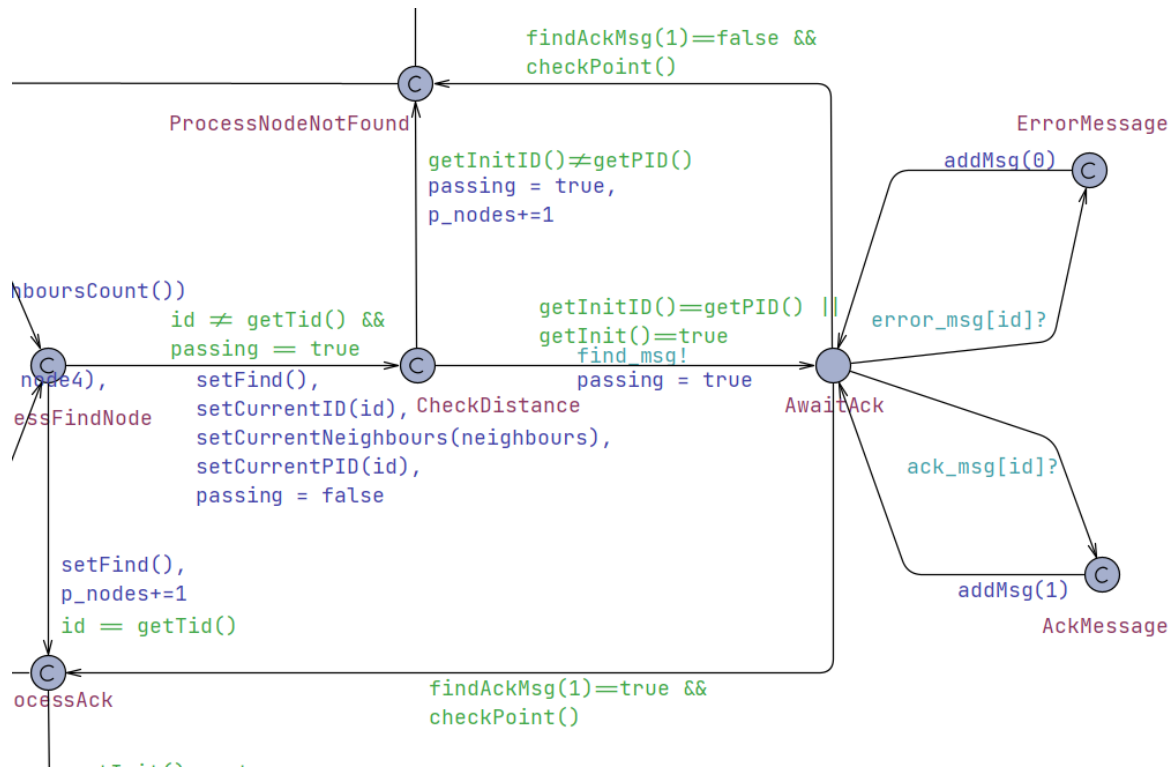


FIGURE 4.7: Forth part of Node template

Verification and Formal Analysis

In this chapter, we check and discuss simple reachability properties, safety properties, liveness properties, and for absence of Deadlock-free Property.

5.1 Deadlock-free Property

In order to make sure the whole system can execute without technical problems, a Deadlock-free Property is important so that the system will not interrupt during the execution. The CTL provides the key word *deadlock* in UPPAAL to achieve that:

- $A[] \text{ not deadlock}$: This path formulae guarantees that the model will not progress into the deadlock situation.

The following execution result can be seen in this figure (Figure 5.1).

5.2 Reachability Properties

The simple reachability properties check if the specified location is reachable or not in the system. First of all, in the example, the UAV U_1 will pass N_1 then N_3 then N_5 and finally reach N_{25} . Thus, the location *ProcessReportLocation* should be reachable for N_1 , N_3 and N_5 , and location *ProcessAck*

```
A[] not deadlock
验证费时/kernel费时/总费时: 2.172s / 0.016s / 2.184s.
常驻内存/虚拟内存的使用峰值: 188,176KB / 405,440KB.
满足该性质.
```

FIGURE 5.1: The result of executing Deadlock-free Properties

should be reachable for N_{25} . Besides, N_2 , N_4 and N_6 are not target node branches so that they will not transit to location *ProcessReportLocation*. As for the template UAV, U_1 will never reach the location *Terminate* and will keep executing to reach the location *processAck* for no deadlock property. And for the template WareHouse, WH will never reach the location *ProcessError*, but to reach the location *ProcessReport* for each iteration. Here is the summary:

- $E <> \text{node1.ProcessReportLocation}$: This path formulae guarantees that "There exists at least one execution path where eventually, N_1 reaches the location *ProcessReportLocation*."
- $E <> \text{node3.ProcessReportLocation}$: This path formulae guarantees that "There exists at least one execution path where eventually, N_3 reaches the location *ProcessReportLocation*."
- $E <> \text{node5.ProcessReportLocation}$: This path formulae guarantees that "There exists at least one execution path where eventually, N_5 reaches the location *ProcessReportLocation*."
- $E <> \text{node25.ProcessReportLocation}$: This path formulae guarantees that "There exists at least one execution path where eventually, N_{25} reaches the location *ProcessAck*."
- $A[] \text{ not node2.ProcessReportLocation}$: This path formulae guarantees that "In all possible execution paths, it is always true that N_2 is not in the *ProcessReportLocation* location."
- $A[] \text{ not node4.ProcessReportLocation}$: This path formulae guarantees that "In all possible execution paths, it is always true that N_4 is not in the *ProcessReportLocation* location."
- $A[] \text{ not node6.ProcessReportLocation}$: This path formulae guarantees that "In all possible execution paths, it is always true that N_6 is not in the *ProcessReportLocation* location."
- $A[] \text{ not UAV.Terminate}$: This path formulae guarantees that "In all possible execution paths, it is always true that U_1 is not in the *Terminate* location."
- $E <> \text{UAV.processAck}$: This path formulae guarantees that "There exists at least one execution path where eventually, U_1 reaches the location *processAck*."
- $E <> \text{WareHouse.ProcessReport}$: This path formulae guarantees that "There exists at least one execution path where eventually, WH reaches the location *ProcessReport*."
- $A[] \text{ not WareHouse.ProcessError}$: This path formulae guarantees that "In all possible execution paths, it is always true that WH is not in the *ProcessError* location."

The following execution result can be seen in this figure (Figure 5.2).

```

E<> node1.ProcessReportLocation
验证费时/kernel费时/总费时: 0.078s / 0.031s / 0.112s.
常驻内存/虚拟内存的使用峰值: 132,840KB / 293,668KB.
满足该性质.
E<> node3.ProcessReportLocation
验证费时/kernel费时/总费时: 0.719s / 0.031s / 0.751s.
常驻内存/虚拟内存的使用峰值: 137,132KB / 302,480KB.
满足该性质.
E<> node5.ProcessReportLocation
验证费时/kernel费时/总费时: 1.421s / 0s / 1.422s.
常驻内存/虚拟内存的使用峰值: 160,804KB / 350,088KB.
满足该性质.
E<> node25.ProcessAck
验证费时/kernel费时/总费时: 0.766s / 0s / 0.779s.
常驻内存/虚拟内存的使用峰值: 164,540KB / 357,804KB.
满足该性质.
A[] not node2.ProcessReportLocation
验证费时/kernel费时/总费时: 1.453s / 0s / 1.446s.
常驻内存/虚拟内存的使用峰值: 96,952KB / 220,752KB.
满足该性质.
A[] not node4.ProcessReportLocation
验证费时/kernel费时/总费时: 1.469s / 0s / 1.475s.
常驻内存/虚拟内存的使用峰值: 114,596KB / 256,912KB.
满足该性质.
A[] not node6.ProcessReportLocation
验证费时/kernel费时/总费时: 1.453s / 0s / 1.454s.
常驻内存/虚拟内存的使用峰值: 132,556KB / 293,120KB.
满足该性质.
A[] not Warehouse.ProcessError
验证费时/kernel费时/总费时: 1.39s / 0.031s / 1.455s.
常驻内存/虚拟内存的使用峰值: 108,884KB / 383,244KB.
满足该性质.
E<> Warehouse.ProcessReport
验证费时/kernel费时/总费时: 0.094s / 0s / 0.106s.
常驻内存/虚拟内存的使用峰值: 108,892KB / 383,260KB.
满足该性质.
E<> UAV.processAck
验证费时/kernel费时/总费时: 0.078s / 0s / 0.077s.
常驻内存/虚拟内存的使用峰值: 100,048KB / 365,740KB.
满足该性质.
A[] not UAV.Terminate
验证费时/kernel费时/总费时: 1.453s / 0s / 1.457s.
常驻内存/虚拟内存的使用峰值: 129,292KB / 424,332KB.
满足该性质.

```

FIGURE 5.2: The result of executing Reachability Properties

5.3 Safety Properties

The safety property checks for a bad situation will never happen in the system. In this example, U_1 will only receives *loc_msg* from initial node and start moving from initial node to the target node. In order to make sure that every node has replied to the finding process, no node can still stay at the location *AwaitAck* when U_1 starts moving. The following safety properties must hold in this example for all reachable states. Here is the summary:

- $A[] \text{ not } (UAV.Moving \text{ and } node1.AwaitAck)$: This path formulae guarantees that "In all possible execution paths, it is always true that the system is never in a state where U_1 is in *Moving* and N_1 is in *AwaitAck* at the same time."
- $A[] \text{ not } (UAV.Moving \text{ and } node2.AwaitAck)$: This path formulae guarantees that "In all possible execution paths, it is always true that the system is never in a state where U_1 is in *Moving* and N_2 is in *AwaitAck* at the same time."
- $A[] \text{ not } (UAV.Moving \text{ and } node3.AwaitAck)$: This path formulae guarantees that "In all possible execution paths, it is always true that the system is never in a state where U_1 is in *Moving* and N_3 is in *AwaitAck* at the same time."
- $A[] \text{ not } (UAV.Moving \text{ and } node7.AwaitAck)$: This path formulae guarantees that "In all possible execution paths, it is always true that the system is never in a state where U_1 is in *Moving* and N_7 is in *AwaitAck* at the same time."
- $A[] \text{ not } (UAV.Moving \text{ and } node8.AwaitAck)$: This path formulae guarantees that "In all possible execution paths, it is always true that the system is never in a state where U_1 is in *Moving* and N_8 is in *AwaitAck* at the same time."

The following execution result can be seen in this figure (Figure 5.3).

5.4 Liveness Properties

The following Liveness Properties must hold in this example for all reachable states. These path formulae are used to test whether the function *reset()* works to reset the variables of nodes as default after each iteration of the finding process. Here is the summary:

- $A <> node1.find == false$: this path formulae guarantees that "In all possible execution paths, it is eventually true that the boolean *find* in N_1 becomes false."

```

A[] not (UAV.Moving and node2.AwaitAck)
验证费时/kernel费时/总费时: 1.437s / 0.047s / 1.501s.
常驻内存/虚拟内存的使用峰值: 69,616KB / 383,168KB.
满足该性质.
A[] not (UAV.Moving and node1.AwaitAck)
验证费时/kernel费时/总费时: 1.375s / 0.047s / 1.449s.
常驻内存/虚拟内存的使用峰值: 87,696KB / 419,760KB.
满足该性质.
A[] not (UAV.Moving and node3.AwaitAck)
验证费时/kernel费时/总费时: 1.438s / 0.016s / 1.448s.
常驻内存/虚拟内存的使用峰值: 105,752KB / 456,068KB.
满足该性质.
A[] not (UAV.Moving and node7.AwaitAck)
验证费时/kernel费时/总费时: 1.422s / 0s / 1.444s.
常驻内存/虚拟内存的使用峰值: 123,892KB / 492,784KB.
满足该性质.
A[] not (UAV.Moving and node8.AwaitAck)
验证费时/kernel费时/总费时: 1.422s / 0.031s / 1.45s.
常驻内存/虚拟内存的使用峰值: 142,224KB / 529,908KB.
满足该性质.

```

FIGURE 5.3: The result of executing safety properties

- $A <> node3.find == false$: this path formulae guarantees that "In all possible execution paths, it is eventually true that the boolean $find$ in N_3 becomes false."
- $A <> node5.find == false$: this path formulae guarantees that "In all possible execution paths, it is eventually true that the boolean $find$ in N_5 becomes false."
- $A <> node25.find == false$: this path formulae guarantees that "In all possible execution paths, it is eventually true that the boolean $find$ in N_{25} becomes false."

The following execution result can be seen in this figure (Figure 5.4).

```
A<> node1.find==false
验证费时/kernel费时/总费时: 0s / 0s / 0.001s.
常驻内存/虚拟内存的使用峰值: 131,160KB / 508,556KB.
满足该性质.
A<> node3.find==false
验证费时/kernel费时/总费时: 0s / 0s / 0.002s.
常驻内存/虚拟内存的使用峰值: 127,372KB / 499,932KB.
满足该性质.
A<> node25.find==false
验证费时/kernel费时/总费时: 0s / 0s / 0.001s.
常驻内存/虚拟内存的使用峰值: 127,404KB / 500,000KB.
满足该性质.
A<> node5.find==false
验证费时/kernel费时/总费时: 0s / 0s / 0.001s.
常驻内存/虚拟内存的使用峰值: 127,420KB / 500,024KB.
满足该性质.
```

FIGURE 5.4: The result of executing liveness properties

Conclusion

In this thesis, I designed the scenario of a signal beacons-based UAV delivery system based on the research of MANET and smart cities, used the formal expression ω -calculus to describe and represent the whole process of UAV delivery system. Then, I employed the timed model checker UPPAAL to model and verify the delivery system process. Based on the model of UAV delivery system, there are several main properties, which include Deadlock-free Property, Reachability Properties, Safety Properties and Liveness Properties, have been verified using path formulae to make sure that the whole delivery system can execute normally and with correct output results.

However, due to the limitation of time and knowledge on the details of MANET and algorithm design, there are still many directions and applications that can be applied to expand the functionalities and usage upon this project. Here is the summary of future works:

- This model can only apply one UAV to interact with the signal base station system and works on delivery tasks. If I want to apply multiple delivery UAVs for more complex delivery tasks, the system cannot handle it because of the high density of message transition, since the simple signal base station cannot handle multiple processes at the same time with lots of messages and information. Hence, in the future, the system can be optimized and updated to help handle multiple delivery UAVs, which can significantly increase the delivery efficiency.
- This model does not apply cyber security algorithms to protect the whole system. Since the UAV cannot communicate directly with the warehouse and the company during the delivery task, the UAV is easy to be hijacked and both delivery UAVs and parcels may be lost. Hence, in the future, more security data structures such as block chain can be applied for both the delivery UAV and the signal base station to help defend common cyber attacks.
- In order to save more power for UAV delivering, this system applies dead reckoning navigation for the delivery UAV. However, dead reckoning may cause unpredicted navigation problems

for UAV, as it may not reach the communication range with signal base stations. Thus, a better power-friendly navigation can be applied for delivery UAVs for more stable delivery process.

Bibliography

- Gerd Behrmann, Alexandre David, and Kim G. Larsen. 2004. *A Tutorial on Uppaal*, pages 200–236. Springer Berlin Heidelberg, Berlin, Heidelberg.
- Sathyanarayanan Chandrasekharan, Karina Gomez, Akram Al-Hourani, Sithamparanathan Kandeepan, Tinku Rasheed, Leonardo Goratti, Laurent Reynaud, David Grace, Isabelle Bucaille, Thomas Wirth, and Sandy Allsopp. 2016. Designing and implementing future aerial communication networks. *IEEE Communications Magazine*, 54(5):26–34.
- Vikas Hassija, Vinay Chamola, Adhar Agrawal, Adit Goyal, Nguyen Cong Luong, Dusit Niyato, Fei Richard Yu, and Mohsen Guizani. 2021. Fast, reliable, and secure drone communication: A comprehensive survey. *IEEE Communications Surveys Tutorials*, 23(4):2802–2832.
- Jin Lv, Yongxin Zhao, Xi Wu, Yongjian Li, and Qiang Wang. 2021. Formal analysis of tsn scheduler for real-time communications. *IEEE Transactions on Reliability*, 70(3):1286–1294.
- Nader Mohamed, Jameela Al-Jaroodi, Imad Jawhar, Ahmed Idries, and Farhan Mohammed. 2020. Unmanned aerial vehicles applications in future smart cities. *Technological Forecasting and Social Change*, 153:119293.
- Donatus Ebere Okonta and Vladimir Vukovic. 2024. Smart cities software applications for sustainability and resilience. *Heliyon*, 10(12):e32654.
- Haroon Rashid and Ashok Kumar Turuk. 2015. Dead reckoning localisation technique for mobile wireless sensor networks. *IET Wireless Sensor Systems*, 5(2):87–96.
- Carlos Andrés Tavera Romero, Jesús Hamilton Ortiz, Luis Carlos Rodríguez Timaná, Fernando Vélez Varela, Andrés Hernando Aristizábal Montufar, and Javier Gamboa-Cruzado. 2025. Fanet and manet, a support and composition relationship. *Computers, Materials and Continua*, 82(2):1699–1732.
- Anu Singh, C.R. Ramakrishnan, and Scott A. Smolka. 2010. A process calculus for mobile ad hoc networks. *Science of Computer Programming*, 75(6):440–469.